

Lab Assignment 02 - Pretrained Model

Course Name: Deep Learning Lab

Lab Title: Research Paper Implementation with Pre-trained Model

Research Paper Title: Comparison of Pre-Trained Models Using Transfer Learning for Detecting Plant Disease

Date of Submission: []

Group Members:

Sanket Mane - 202201040137

Dhiraj Rathod - 202201040139

Github Link: https://github.com/sanketdattatraymane/DL_Assignment.git

Research Paper Link: <https://ieeexplore.ieee.org/abstract/document/9397098>

Research Paper Summary

Title: Comparison of Pre-Trained Models Using Transfer Learning for Detecting Plant Disease

Authors:

Bincy Chellapandi, M. Vijayalakshmi, Shalu Chopra

Published In: 2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)

Dataset Details

Dataset Used: PlantVillage Dataset (from SP Mohanty's GitHub)

Number of Plant Species: 14

Number of Classes: 38 (different types of plant diseases)

Total Images: 54,306 (256x256 RGB images)

Training/Testing Split: 80% training, 20% testing

Augmentation Techniques: Rotating, shearing, flipping, zooming, brightness adjustment (via Keras ImageDataGenerator)

Model Selection

Model	Accuracy	Precision	Recall	F1-Score	
-----	-----	-----	-----	-----	
InceptionV3	97%	96%	94%	95%	
InceptionResNetV2	98%	97%	96%	97%	
ResNet50	93%	88%	87%	86%	
VGG16	99%	95%	95%	95%	
VGG19	98%	100%	91%	95%	
MobileNet	98%	97%	95%	95%	
MobileNetV2	95%	84%	86%	85%	
DenseNet	99%	98%	98%	98%	
Custom CNN	87%	—	—	—	

Our Implementation

Dataset Preparation:

Dataset Used: PlantVillage Dataset (38 disease classes, 14 plant species)

Image Resolution: 256x256 pixels (RGB)

Train-Test-Validation Split:

Training: 70%

Validation: 15%

Testing: 15%

Model Architecture & Training Strategy:

Modified Model Architecture:

Added custom dense layers with dropout and batch normalization

Used GlobalAveragePooling2D instead of flattening to reduce overfitting

Final layer: Softmax activation for multi-class classification

Transfer Learning:

`base_model.trainable = False` — Frozen pre-trained layers to retain learned features

Hyperparameter Tuning:

Learning Rate: Experimented with 0.0001

Batch Size: 16

Epochs: Ran experiments with 10 epochs (early stopping used)

Optimizer: Adam (with weight decay for regularization)

Key Findings:

EfficientNet provided the best balance of speed and accuracy.

VGG19 delivered high accuracy but was computationally heavier.

MobileNet is a good option for lightweight, faster training but sacrifices a bit of accuracy.

ResNet50 struggled slightly with class imbalance but was still robust for feature extraction

General CNN Models Overview

1. VGG19

Developed by: Visual Graphics Group (VGG), Oxford University (2014)

Architecture: 19 layers (16 convolutional + 3 fully connected)

Key Features:

Stacks small 3x3 convolution filters.

Simple, uniform architecture.

Learns rich, hierarchical features.

Use Cases: Image classification, object detection, style transfer.

Limitation: High parameter count, resource-intensive.

2. ResNet50 (Residual Networks)

Developed by: Microsoft Research (2015)

Architecture: 50 layers with skip connections.

Key Features:

Skip connections prevent vanishing gradients.

Enables very deep networks without performance loss.

Strong feature extraction capabilities.

Use Cases: Object recognition, medical imaging, image segmentation.

Limitation: Larger size, needs more computation than lightweight models.

3. MobileNetV3

Developed by: Google (2019)

Architecture: Designed for mobile and edge devices.

Key Features:

Lightweight and fast.

Uses depthwise separable convolutions.

Optimized with squeeze-and-excitation blocks and hard-swish activation.

Use Cases: Real-time classification, object tracking, mobile apps.

Limitation: Lower accuracy compared to larger models.

4. EfficientNetB0

Developed by: Google Brain (2019)

Architecture: Scales width, depth, and resolution using compound scaling.

Key Features:

High accuracy with fewer parameters.

Scalable model family (B0 to B7).

State-of-the-art performance for classification tasks.

Use Cases: Fine-grained classification, image recognition, complex vision tasks.

Limitation: More complex to tune than simpler architectures.

CODE IMPLEMENTATION

Step 1: Configuring the environment setup

Configuration environment

```
import os
```

```
os.environ['KAGGLE_USERNAME'] = "user" # username from the json file
```

```
os.environ['KAGGLE_KEY'] = "key" # key from the json file
```

```
!kaggle datasets download -d noulam/tomato
```

```
!unzip tomato.zip -d tomato
```

Step 2: Data Preprocessing

import dependencies

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import cv2
```

```
import os
```

```
import time
```

```
import random
```

```
import torch
```

```
import torchvision
```

```
from torch import nn

from torch.utils.data import DataLoader

from torch.optim import Adam


from torchvision import transforms, datasets

from torchvision.models import resnet50, ResNet50_Weights

from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights


from PIL import Image

from pathlib import Path

from tqdm.auto import tqdm

from timeit import default_timer as timer

from typing import Dict, List

path = "tomato/New Plant Diseases Dataset(Augmented)"

os.listdir(path)

train_path = os.path.join(path, "train")

print(os.listdir(train_path))

print("*"*100)

test_path = os.path.join(path, "valid")

print(os.listdir(test_path))

from glob import glob

folders = glob("tomato/New Plant Diseases Dataset(Augmented)/train/*")

folders

import matplotlib.pyplot as plt
```

```
train1 = os.path.join(train_path,"Tomato___Bacterial_spot/00416648-be6e-4bd4-bc8d-82f43f8a7240___GCREC_Bact.Sp 3110.JPG")
```

```
plt.imshow(plt.imread(train1))
```

```
plt.title("Bacterial Spot")
```

```
from tensorflow.keras.layers import Input, Lambda, Dense, Flatten
```

```
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.applications.inception_v3 import InceptionV3
```

```
from tensorflow.keras.preprocessing import image
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
from tensorflow.keras.models import Sequential
```

Trying vgg16 separately

```
SIZE = [128, 128]
```

```
from tensorflow.keras.applications.vgg16 import VGG16
```

```
from tensorflow.keras.applications.vgg19 import VGG19
```

```
vgg19 = VGG19(input_shape=SIZE + [3], weights="imagenet",  
include_top=False)
```

```
# The weights of that layer will not be updated during the training process
```

```
for layer in vgg19.layers:
```

```
    layer.trainable = False
```

```
# Take the output of VGG16 & flatten the multi-dimensional neuron tensor into  
single dimension vector
```

```
x = Flatten()(vgg19.output)
```

```
prediction = Dense(len(folders), activation="softmax")(x)
```



```
modelvgg = Model(inputs=vgg19.input, outputs=prediction)
modelvgg.summary()
```

Step 3: Data Preparing

Used to configure the model for training

loss="categorical_crossentropy" --> it tells the model how wrong its predictions are. The goal of training is to minimize this loss

metrics=["accuracy"]--> specifies that the "accuracy" metric should be calculated and displayed during training.

```
modelvgg.compile(loss="categorical_crossentropy", metrics=["accuracy"],
optimizer="adam")
```

```
train_datagen_vgg19 = ImageDataGenerator(rescale=1./255)
```

```
test_datagen_vgg19 = ImageDataGenerator(rescale=1./255)
```

method that generates batches of augmented/preprocessed image data from a directory.

```
# resizes all the images to 128x128 pixels. [VGG model needs consistent size]
```

```
training_set_vgg19 = train_datagen_vgg19.flow_from_directory(train_path,  
                                                             target_size=(128, 128),  
                                                             batch_size=16,  
                                                             class_mode="categorical", shuffle=True)
```

[illegible]

```
class_mode="categorical", shuffle=False)
```

```
import tensorflow as tf
```

```
callback = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3)
```

```
# r_vg19 = modelvgg.fit(training_set_vgg19,  
#                       validation_data=testing_set_vgg19,  
#                       epochs=50,  
#                       callbacks=[callback]  
#                       )
```

Step 5: Building the models

```
from tensorflow.keras.applications import VGG19, ResNet50,  
MobileNetV3Small, EfficientNetB0
```

```
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
```

```
from tensorflow.keras.optimizers import Adam
```

```
def build_model(base_model, num_classes):
```

```
    base_model.trainable = False # Freeze the pretrained layers
```

```
    x = base_model.output
```

```
    x = GlobalAveragePooling2D()(x)
```

```
    x = Dense(256, activation='relu')(x)
```

```
    x = Dropout(0.5)(x)
```

```
output = Dense(num_classes, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)

return model
```

```
models = {

    "VGG19": VGG19(weights="imagenet", include_top=False,
input_shape=(128, 128, 3)),

    "ResNet50": ResNet50(weights="imagenet", include_top=False,
input_shape=(128, 128, 3)),

    "MobileNetV3": MobileNetV3Small(weights="imagenet", include_top=False,
input_shape=(128, 128, 3)),

    "EfficientNet": EfficientNetB0(weights="imagenet", include_top=False,
input_shape=(128, 128, 3))

}
```

```
from sklearn.metrics import classification_report, accuracy_score,
precision_score, recall_score, f1_score

import numpy as np

import pandas as pd

import json
```

```
def evaluate_model(model, test_set, model_name):

    y_pred = model.predict(test_set)

    y_pred_classes = np.argmax(y_pred, axis=1)

    y_true = test_set.classes
```

```
# Calculate metrics
```

```
accuracy = accuracy_score(y_true, y_pred_classes)
```

```
precision = precision_score(y_true, y_pred_classes, average='weighted')
```

```
recall = recall_score(y_true, y_pred_classes, average='weighted')
```

```
f1 = f1_score(y_true, y_pred_classes, average='weighted')
```

```
loss, _ = model.evaluate(test_set)
```

```
# Save classification report
```

```
class_report = classification_report(y_true, y_pred_classes,  
target_names=test_set.class_indices.keys(), output_dict=True)
```

```
# Save results to a dictionary
```

```
result = {  
    "Model": model_name,  
    "Accuracy": accuracy * 100,  
    "Precision": precision * 100,  
    "Recall": recall * 100,  
    "F1-Score": f1 * 100,  
    "Loss": loss,  
    "Classification Report": class_report  
}
```

```
# Save results to JSON
```

```
with open(f"{model_name}_evaluation.json", "w") as f:
```

```
    json.dump(result, f, indent=4)
```

```
return result
```

```
# results = []
```

```
# for model_name, base_model in models.items():
```

```
#     print(f"\nTraining {model_name}...\n")
```

```
#     model = build_model(base_model, num_classes=10)
```

```
#     model.compile(optimizer=Adam(learning_rate=0.001),  
loss='categorical_crossentropy', metrics=['accuracy'])
```

```
#     history = model.fit(trainning_set_vgg19, epochs=5,  
validation_data=testing_set_vgg19)
```

```
#     # Evaluate and save results
```

```
#     result = evaluate_model(model, testing_set_vgg19, model_name)
```

```
#     results.append(result)
```

```
#     # Save model
```

```
#     model.save(f"{model_name}_plant_disease_model.h5")
```

Step 4: Train the Models

```
results = []
```

```
for model_name, base_model in models.items():
```

```
    print(f"\nTraining {model_name}...\n")
```

```

model = build_model(base_model, num_classes=10)

model.compile(optimizer=Adam(learning_rate=0.0001),
loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(training_set_vgg19, epochs=7,
validation_data=testing_set_vgg19)

# Evaluate and save results

result = evaluate_model(model, testing_set_vgg19, model_name)
results.append(result)

# Save model

model.save(f'{model_name}_plant_disease_model.h5')

```

Step 6: Result & Interpretation

```

# Create and save results as CSV

result_df = pd.DataFrame([{'k': v for k, v in res.items() if k != "Classification
Report"} for res in results])

result_df.to_csv("model_comparison_results.csv", index=False)

# Print the results

print(result_df)

# Load previous results

loaded_results = pd.read_csv("model_comparison_results.csv")
print(loaded_results)

```

```
# Load classification report for a specific model
```

```
with open("VGG19_evaluation.json") as f:
```

```
    vgg19_report = json.load(f)
```

```
    print(vgg19_report['Classification Report'])
```

```
from sklearn.metrics import classification_report, accuracy_score,  
precision_score, recall_score, f1_score
```

```
import numpy as np
```

```
import pandas as pd
```

```
import json
```

```
def evaluate_model(model, test_set, model_name):
```

```
    y_pred = model.predict(test_set)
```

```
    y_pred_classes = np.argmax(y_pred, axis=1)
```

```
    y_true = test_set.classes
```

```
# Calculate metrics
```

```
accuracy = accuracy_score(y_true, y_pred_classes)
```

```
precision = precision_score(y_true, y_pred_classes, average='weighted')
```

```
recall = recall_score(y_true, y_pred_classes, average='weighted')
```

```
f1 = f1_score(y_true, y_pred_classes, average='weighted')
```

```
loss, _ = model.evaluate(test_set)
```

```
# Save classification report
```

```
class_report = classification_report(y_true, y_pred_classes,  
target_names=test_set.class_indices.keys(), output_dict=True)
```

```
# Save results to a dictionary
```

```
result = {  
    "Model": model_name,  
    "Accuracy": accuracy * 100,  
    "Precision": precision * 100,  
    "Recall": recall * 100,  
    "F1-Score": f1 * 100,  
    "Loss": loss,  
    "Classification Report": class_report  
}
```

```
# Save results to JSON
```

```
with open(f"{model_name}_evaluation.json", "w") as f:  
    json.dump(result, f, indent=4)
```

```
return result
```

```
import matplotlib.pyplot as plt
```

```
def plot_metrics(history, model_name):
```

```
    plt.figure(figsize=(12, 6))
```

```
    plt.subplot(1, 2, 1)
```



```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Test Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title(f'{model_name} - Accuracy')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Test Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title(f'{model_name} - Loss')
plt.legend()
plt.show()
```

```
plot_metrics(history,"VGG19")
```

Declaration

I, Sanket Mane, confirm that the work submitted in this assignment is my own and has been completed following academic integrity guidelines. The code is uploaded on my GitHub repository account, and the repository link is provided below:

GitHub Repository Link:

https://github.com/sanketdattatraymane/DL_Assignment.git

Signature: Sanket Mane